

Practical week 2

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using fork() system call.
- b) Read the user input as shell command.
- c) Use a system call exec() to execute the shell command.
- d) Use a wait() system call for waiting a parent process.
- e) Display the PIDs of parent and child processes.

```
saikrishna@DESKTOP-LGFVDTJ:~$ ./Practical
Enter a Linux command: sh

Parent Process

Parent PID: 798
Child Process
Child PID : 799
Child PID : 799
Parent PID: 798
Executing command: sh

$ |
```

Practical week 2

```
Child process completed.
saikrishna@DESKTOP-LGFVD1:~$ cat Practical.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    else if (pid == 0)
    {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        printf("Executing command: %s\n\n", command);

        execl("/bin/sh", "sh", "-c", command, (char *)NULL);

        perror("exec failed");
        exit(1);
    }

    else
    {
        // Parent process
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID : %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}
```

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Practical week 2

Report: How the Operating System Abstracts Hardware Resources

1. Introduction

An Operating System (OS) acts as an interface between **users/applications and computer hardware**. Applications do not usually communicate directly with the CPU, RAM, storage devices, or I/O devices. Instead, they request services from the OS, and the OS manages the underlying hardware.

The main hardware resources considered are:

- CPU
- Memory (RAM)
- Storage
- I/O devices

The OS provides an **abstraction layer** that hides the complexity of physical hardware from application programs.

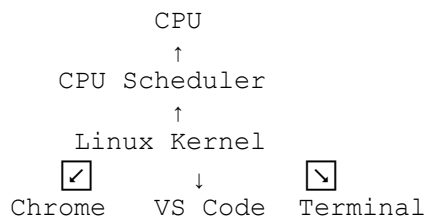
2. CPU Abstraction

The CPU is responsible for executing program instructions. When multiple programs are running, the OS uses a **process scheduler** to decide which process gets CPU time.

For example, if these applications are running:

Chrome
VS Code
Terminal

the OS manages their execution:



The application does not need to know which CPU core is executing its instructions or exactly when the CPU will execute them.

Practical week 2

OS services for CPU

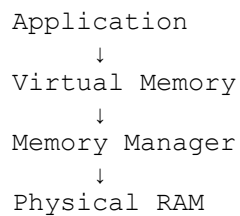
- Process scheduling
- Multitasking
- Context switching
- Process prioritization
- CPU resource allocation

The `lscpu` command provides information about the physical CPU, while `ps` and `top` show how processes are using CPU resources.

3. Memory Abstraction

RAM temporarily stores programs and data that are being used by the CPU.

The OS provides each process with its own **virtual address space**. Therefore, a program can work with memory without knowing the actual physical RAM locations.



For example, two processes may both use an address such as `0x1000`, but the OS maps those virtual addresses to different physical memory locations.

OS services for memory

- Memory allocation
- Memory deallocation
- Virtual memory
- Memory protection
- Paging
- Swap management

This abstraction prevents one process from directly accessing another process's memory.

4. Storage Abstraction

Storage devices include HDDs, SSDs, and USB drives.

Practical week 2

Applications normally do not directly control the physical disk. Instead, they interact with files and directories through the OS's **file system**.

```
Application
  ↓
File Operations
  ↓
File System
  ↓
Linux Kernel
  ↓
Storage Driver
  ↓
SSD / HDD
```

For example, when an application wants to save:

```
student.txt
```

it simply asks the OS to create/write the file. The OS handles the physical details of where the data should be stored on the disk.

OS services for storage

- File creation
- File reading/writing
- Directory management
- File permissions
- Disk management
- Storage-device drivers

Thus, applications work with simple files rather than physical disk sectors.

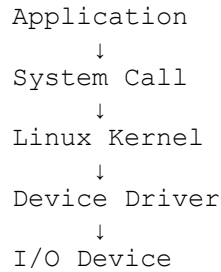
5. I/O Device Abstraction

I/O devices include:

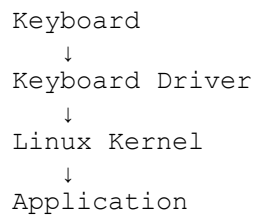
- Keyboard
- Mouse
- Monitor
- Printer
- Network adapter
- USB devices

The OS uses **device drivers** to communicate with these devices.

Practical week 2



For example, when a user presses a key on the keyboard:

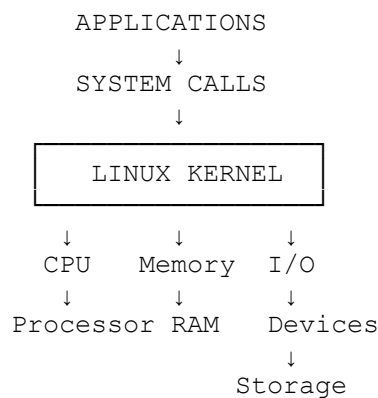


The application does not need to understand the electrical or hardware-level operation of the keyboard.

OS services for I/O

- Device drivers
- Input/output management
- Buffering
- Device allocation
- Interrupt handling
- Hardware communication

6. Overall Hardware Abstraction



The **Linux kernel** is responsible for managing these resources and providing applications with a simpler interface.

7. Conclusion

The Operating System abstracts hardware by hiding the complexity of physical resources and providing convenient services to applications.

- **CPU:** The OS provides process scheduling and multitasking.
- **Memory:** The OS provides virtual memory, allocation, and protection.
- **Storage:** The OS provides files, directories, and file-system services.
- **I/O:** The OS uses device drivers to provide a standard interface to hardware devices.

Therefore, hardware abstraction allows programmers to develop applications without needing to directly manage the physical CPU, RAM, disk sectors, or individual hardware devices.